



RAG REPAIR WORKBENCH

100 cases · faith 0.63

100 cases · faith 0.74

100 cases · faith 0.74

- 1 Upload**
Add a RAGVue eval file
- 2 Diagnose**
Cluster failures into families & slices
- 3 Repair**
Approve a parameterised fix
- 4 Sandbox**
Rerun affected cases with patch
- 5 Verify**
Review before/after delta report

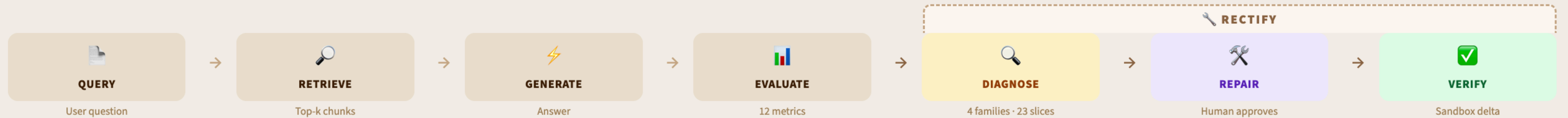
10

Bad chunks / low coverage

Hallucination / drift



Stop debugging RAG case-by-case. Cluster the failures, approve one fix, verify the delta.



 Rule-based root cause inference per repair slice
 23 fine-grained repair slices (R1-R7 · G1-G7 · S1-S3 · Q1-Q3 · A1-A3)
 Developer approval gate before any change is applied
 Before / after met

[Upload / Load](#)
[Case Explorer](#)
[Diagnose](#)
[Repair Lab](#)
[Delta Explorer](#)
[Approval History](#)

Load a RAGVue evaluation file to start diagnosing your RAG pipeline. Rectify clusters failures into **4 families · 23 repair slices**, generates parameterised repair cards for human approval, and runs a before/after sandbox verification — all in one workbench.

- See how Rectify reduces debugging effort

17

Faithfulness 0.63

Retrieval rel. 0.45

Load →

1

Faithfulness 0.74

Retrieval rel. 0.57

Load →

Faithfulness 0.74

Retrieval rel. 0.56

Load →

Drop a RAGVue JSON file here

